PENSE-BÊTE · TECHNIQUE ET COMMANDES

Évolution des architectures applicatives

06-250 PSE 25-27 2-9 L'Évolution des Architectures Applicatives — source : fiche de révision du module (gemma4, 03/11/2026)

À RETENIR

1. L'architecture monolithique centralise les applications et les données en un seul point.
2. Le modèle 3-tiers introduit une couche intermédiaire de logique métier pour isoler la présentation des données.
3. Les microservices découpent une application en services indépendants communiquant via des API.

Évolution des architectures applicatives

Notion	À retenir
Mainframes (1960–1980)	Centralisation absolue via des **terminaux passifs** (affichage/saisie sans calcul). Architecture **monolithique** simplifiant maintenance et sécurité, mais créant des **goulots d'étranglement**.
Client-serveur (1990)	Architecture **2-tiers** : interface au client, données au serveur. Réduit la charge serveur mais complexifie le **déploiement** et la maintenance sur des parcs **hétérogènes**.
Web et recentralisation (1990–2000)	Traitements sur serveurs web, clients via **navigateurs** (clients légers). Standards **HTTP** et **HTML**. Déploiement simplifié, mais rechargement complet de page nécessaire.

Comparaison des modèles d'architecture

	Mainframes	Client-serveur
Répartition des tâches	Centralisation absolue	Répartition (interface client / données serveur)
Rôle du terminal	Passif (affichage/saisie uniquement)	Exploite la puissance des postes de travail
Maintenance	Simplifiée (point unique)	Complexe (déploiement sur chaque poste client)

Évolution des architectures et interopérabilité

Notion	À retenir
Architectures 3-tiers/n-tiers	Séparation **présentation**, **métier** (ex: `J2EE`, `.NET`) et **données**. Le **n-tiers** ajoute **cache** ou **équilibrage de charge**.
Middleware et promesses	Les **Message Oriented Middleware** assurent l'asynchrone. `Node.js` utilise les **promesses** pour gérer les flux sans infrastructures lourdes comme les **ESB**.
Services web	**SOAP** (XML) est rigoureux et typé. **REST** (HTTP/JSON) est **stateless**, plus léger et flexible, favorisant la **scalabilité**.

REST et frameworks modernes

Notion	À retenir
REST (APIs)	Échanges via **URL** et verbes **HTTP** ; format **JSON** privilégié ; mode **stateless** (sans état) pour la répartition de charge.
Clients riches	Les **SPA** chargent le code une fois et utilisent des appels API ; le **Server-Side Rendering** résout les problèmes de **SEO**.
Data exchange	Le **JSON** peut causer sur-récupération ou sous-récupération ; **GraphQL** permet de demander exactement les champs nécessaires en une seule requête.

Comparaison des approches de rendu et d'échange

	Approche A	Approche B
Rendu	SPA : chargement unique, réactivité native, problèmes d'indexation	Server-Side Rendering : génération côté serveur, résout les problèmes SEO
Échange de données	REST : risque de sur-récupération ou sous-récupération de données	GraphQL : demande exacte des champs nécessaires, idéal pour données interconnectées

Microservices et Conteneurisation

Notion	À retenir
Microservices	Services autonomes par **fonction métier** ; communication via **API légères** ou **messages asynchrones** ; améliore la **résilience** mais complexifie les **données distribuées**.
Conteneurisation	Encapsule application et dépendances pour la **portabilité** ; plus **légers** que les VM (**partage du noyau**) ; **Kubernetes** orchestre le déploiement et la **mise à l'échelle**.

Pièges et confusions

- ⚠ Confondre la centralisation des mainframes avec la décentralisation du client-serveur.
- ⚠ Croire que les microservices sont simples à gérer sans une maturité technique élevée.
- ⚠ Oublier que chaque modèle architectural impose un compromis entre maintenance et complexité de déploiement.
- ⚠ Confondre l'évolution des middlewares d'infrastructure lourds avec les solutions de code légères pour l'asynchrone.

Sigles et définitions

Mainframe Ordinateur central puissant utilisé comme épine dorsale informatique.

Middleware Logiciel assurant la communication et l'échange de données entre différentes applications ou couches.

Stateless Chaque requête contient toutes les informations nécessaires sans dépendre d'un contexte serveur.

SPA Application web chargeant le code une seule fois et mettant à jour le contenu dynamiquement.

GraphQL Langage de requête permettant au client de définir précisément la structure des données souhaitées.

Orchestration Automatisation du déploiement et de la gestion des clusters de conteneurs.

Auto-test

1. Quel est le rôle d'un terminal passif ?
2. Quelle est la différence majeure entre SOAP et REST concernant le format ?
3. Quel est l'avantage principal de la conteneurisation par rapport aux VM ?
4. Comment le SSR traite-t-il le contenu d'une page ?
5. Quel mécanisme permet d'éviter les blocages des flux asynchrones sans infrastructure lourde ?

Réponses : **1.** Affichage et saisie sans capacité de calcul propre. · **2.** SOAP utilise XML tandis que REST s'appuie sur JSON. · **3.** Les conteneurs sont plus légers car ils partagent le noyau du système d'exploitation. · **4.** Il génère le contenu côté serveur pour optimiser le référencement. · **5.** Une promesse.

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.